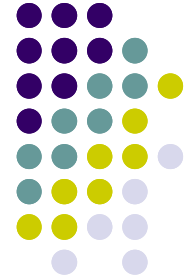


Object Oriented Design Principles

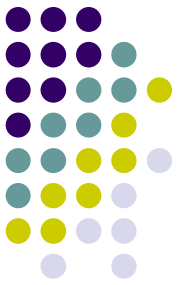


Architecture of Real-Time Systems
SE 456

Ed Keenan

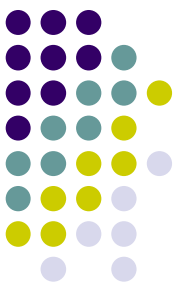
10 March 2021

13.0.8.6.1 Mayan Long Count



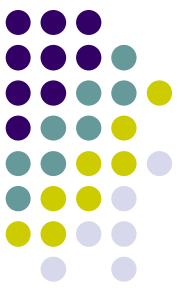
Reference Material

- Reference and Links
 - <https://www.dofactory.com/net/design-patterns>
 - <http://www.oodesign.com/factory-pattern.html>
 - <http://deviq.com/category/principles/>



Software Design Principles

- Set of guidelines
 - Signs of a bad design that should be avoided
 - **Rigidity**
 - It is hard to change because every change affects too many other parts of the system.
 - **Fragility**
 - When you make a change, unexpected parts of the system break.
 - **Immobility**
 - It is hard to reuse in another application because it cannot be disentangled from the current application.



Maximize Cohesion Minimize Coupling

- **Cohesion**

- How closely the operations in a routine are related

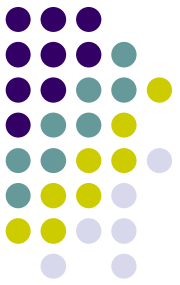
- **Coupling**

- How strongly a routine is related to other routines

- **GOAL**

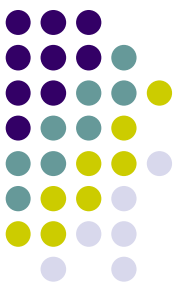
- Create routines with internal integrity
 - Maximize cohesion (strong cohesion)
- Create routines with small, direct, visible, and flexible relations to other routines
 - Minimize coupling (loose coupling)

Open Close Principle



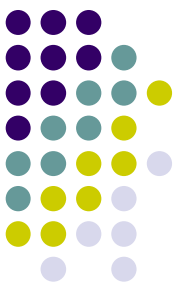
OPEN CLOSED PRINCIPLE

Brain surgery is not necessary when putting on a hat.



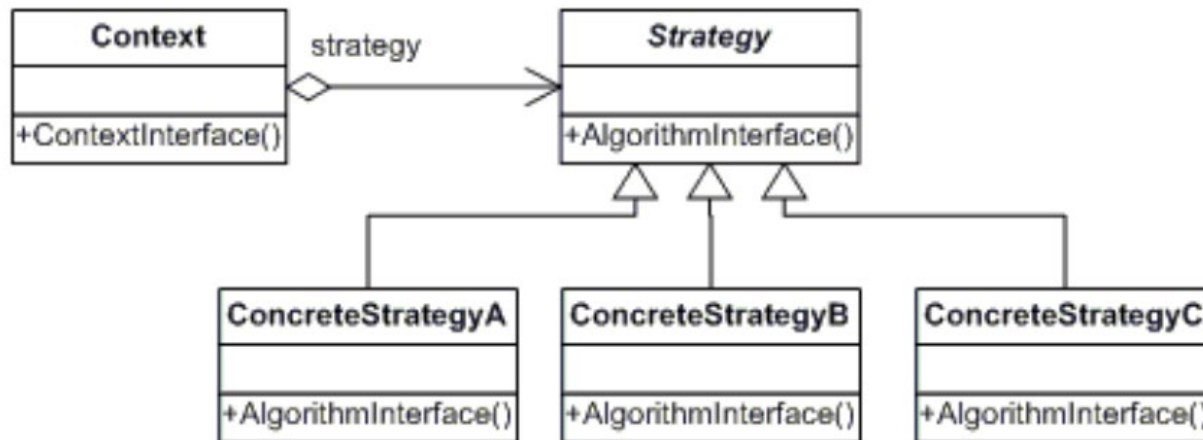
Open Close Principle

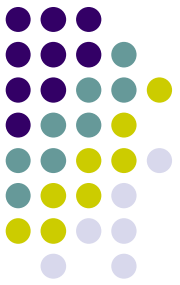
- Intent
 - *Classes should be open for extension but closed for modifications.*
- Open Close Principle is a generic principle.
 - Designing classes
 - To **extend** behavior you **don't** change the class but to **extend** it.
 - Preservers backwards compatibility
- Open Close Principle can be ensured by use of **Abstract Classes** and concrete classes for implementing their behavior.
 - Enforces Concrete Classes extending Abstract Classes **instead** of changing them.



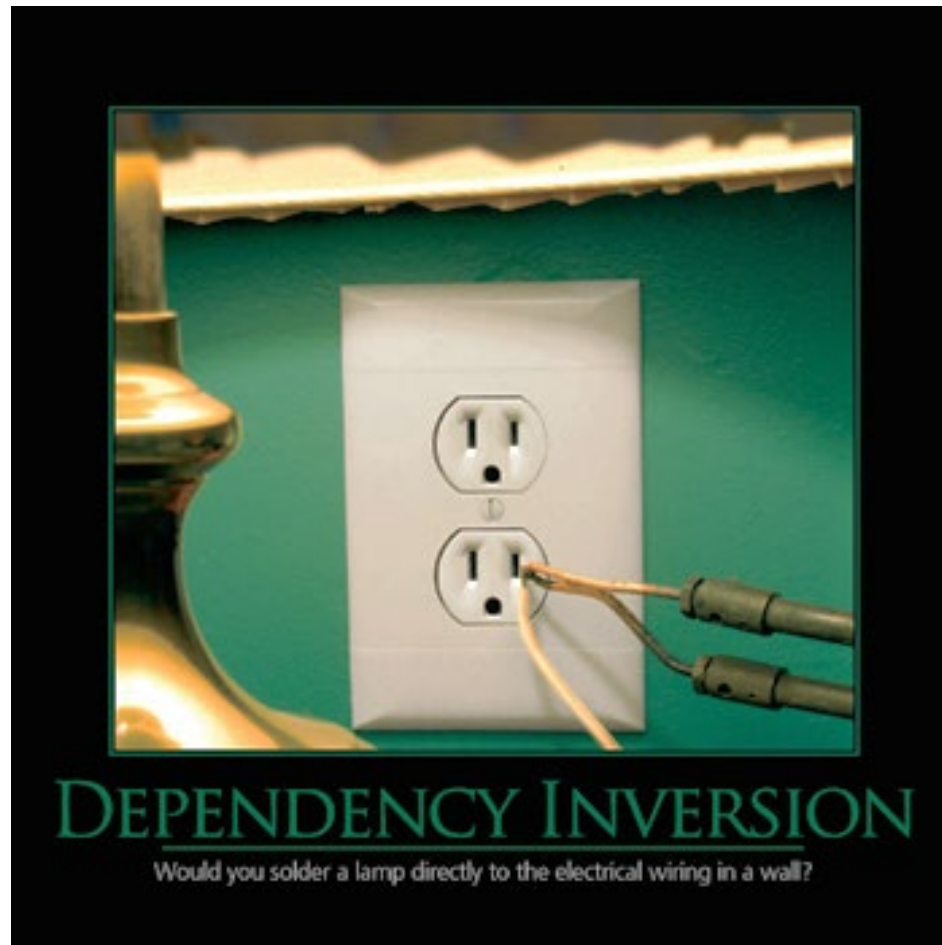
Open Close Principle

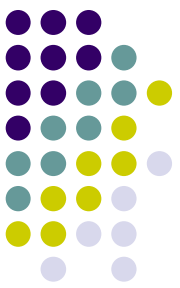
- Strategy pattern
 - Open for extension (**new strategies**)
 - Closed for modifications (**abstract strategy class**)





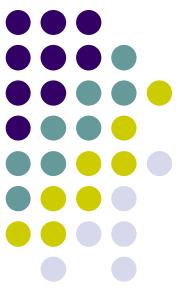
Dependency Inversion Principle



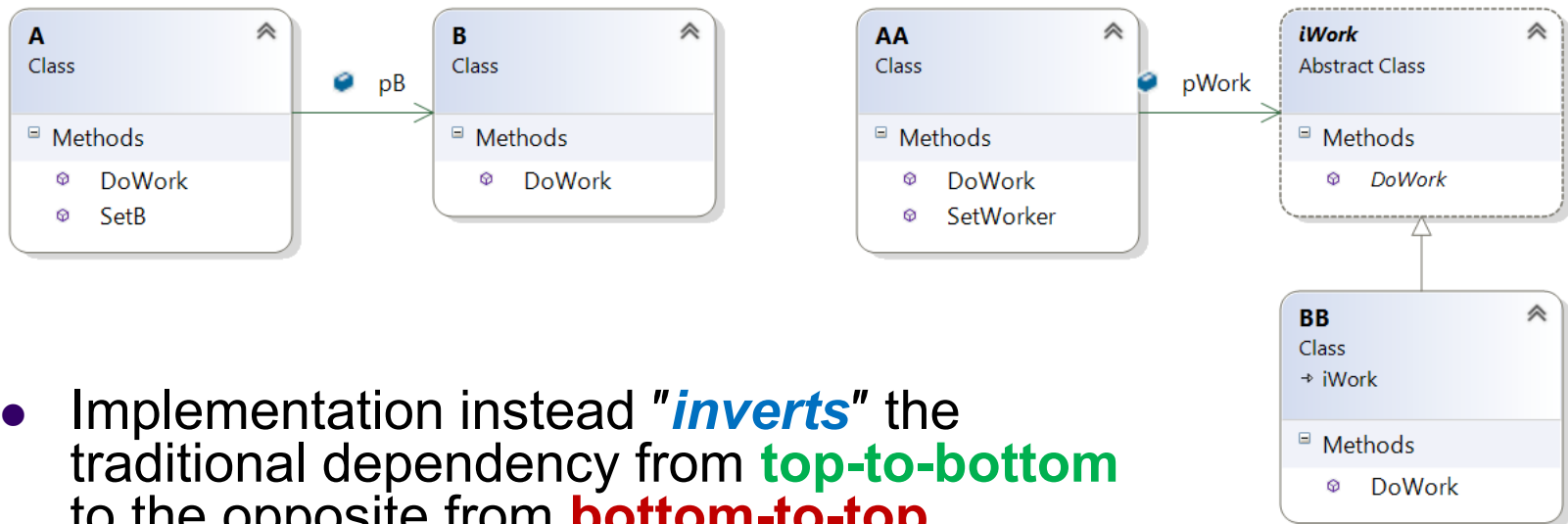


Dependency Inversion Principle

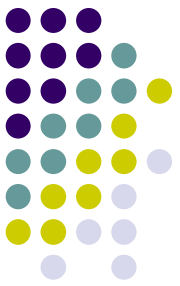
- Intent
 - High-level modules should not depend on low-level modules.
 - Both should depend on abstractions.
 - Abstractions should not depend on details.
 - Details should depend on abstractions.
- Dependency Inversion (Inversion of Control) refers to the way in which the dependencies are realized.
 - Normally when a class need some other class
 - It initializes and holds a direct reference to it.
 - Makes the 2 modules tight coupled.
 - To decouple we provide a hook (property, parameter) and an external module controlling the dependencies will inject the reference to the second one.
 - High Level Classes → Abstraction Layer → Low Level Classes
- Modules can be easily changed by other modules just changing the dependency module.



Dependency Inversion Principle

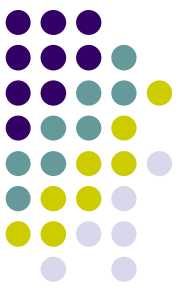


- Implementation instead "*inverts*" the traditional dependency from **top-to-bottom** to the opposite from **bottom-to-top**



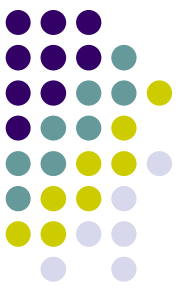
Interface Segregation Principle





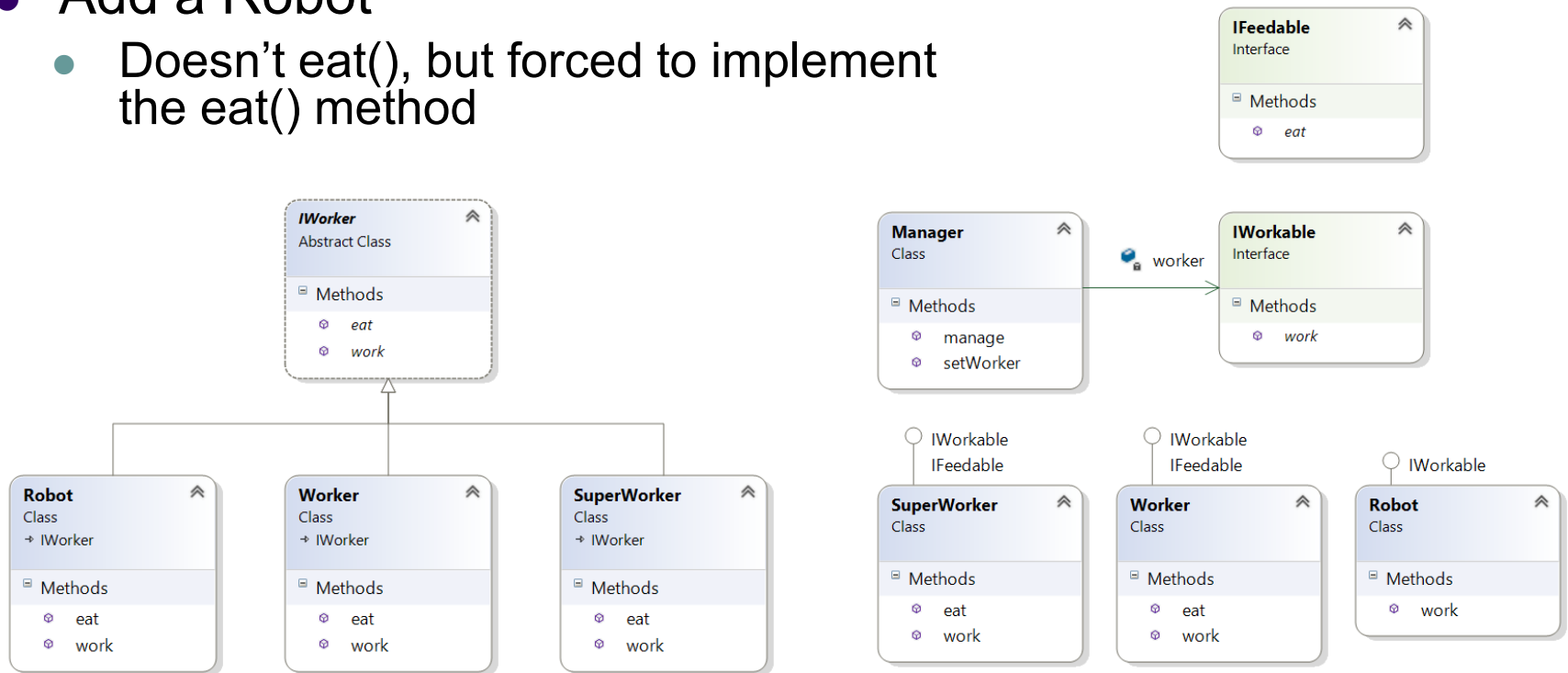
Interface Segregation Principle

- Intent
 - Clients should not be forced to depend upon interfaces that they don't use.
- Describes how to write interfaces
 - Interfaces we should take care to add only methods that should be there.
 - If we add methods that should **not** be there the classes implementing the interface will have to implement those methods as well.
 - Avoid interfaces containing methods that are not specific to it are called **polluted** or **fat interfaces**.
- Do I need all the methods on this interface I'm using?
 - If No, then you might want to use a different interface

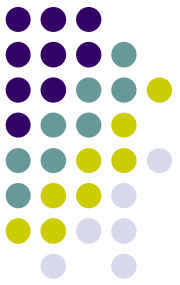


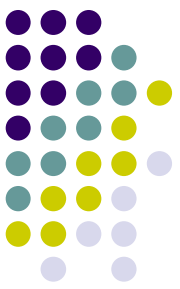
Interface Segregation Principle

- Add a Robot
 - Doesn't eat(), but forced to implement the eat() method



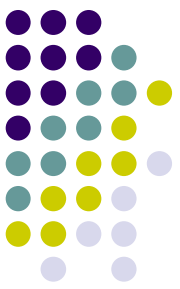
Single Responsibility Principle





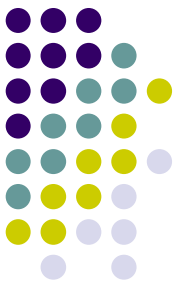
Single Responsibility Principle

- Intent
 - *A class should have only one reason to change.*
- Issue
 - If we have 2 reasons to change for a class
 - We have to split the functionality in two classes.
 - Each class will handle only one responsibility
 - If we need to make one change we are going to make it in the class which handle it.
 - When we need to make a change in a class having more responsibilities the change might affect the other functionality of the classes.
 - A **responsibility** is considered to be one reason to change



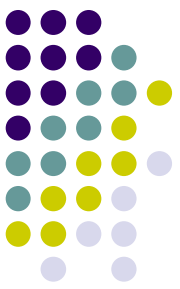
Single Responsibility Principle

- Another way of saying
 - **Gather** together the things that change for the same reasons.
 - **Separate** those things that change for different reasons.
- This is just another way to define cohesion and coupling.
 - We want to increase the cohesion between things that change for the same reasons, and we want to decrease the coupling between those things that change for different reasons.



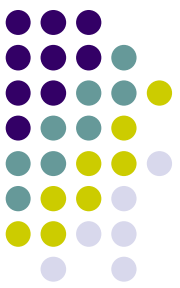
Liskov's Substitution Principle





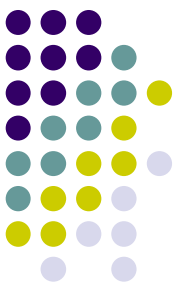
Liskov's Substitution Principle

- Intent
 - *Derived types must be completely substitutable for their base types.*
- Extension of the Open Close Principle in terms of behavior meaning that we must make sure that new derived classes are extending the base classes without changing their **behavior**.
 - The new derived classes should be able to replace the base classes without any change in the code.



Liskov's Substitution Principle

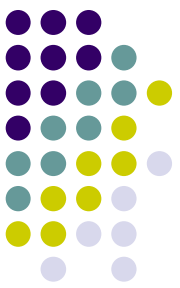
- Take a given class
 - We extend some classes creating some derived classes.
- Must **guarantee** the new derived classes just extend without replacing the functionality of old classes.
 - Otherwise new classes can produce undesired effects
- Liskov's Substitution Principle states that if a program module is using a Base class, then the reference to the Base class can be replaced with a Derived class without affecting the functionality of the program module.



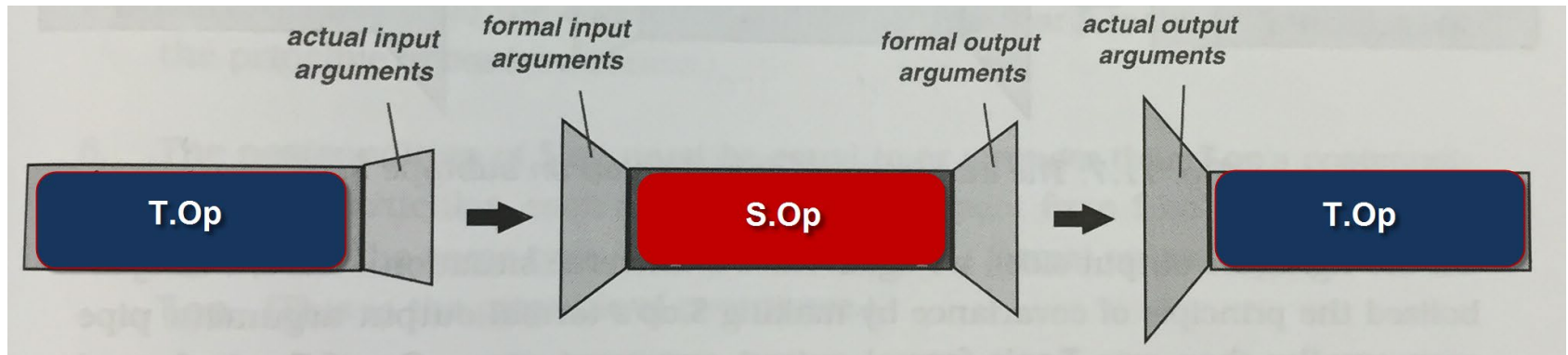
Principle of Contra / Covariance

Conformance

- If **S** is a true subtype of **T**, $\rightarrow$ then **S** must conform to **T**
- Otherwords
 - An object of type **S** can be provided in any context where an object of Type **T** is expected and correctness is still preserved when any accessor operation of the object is executed

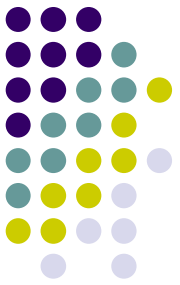


Principle of Contra / Covariance



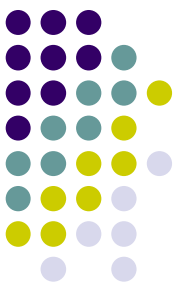
- **Contravariance (input side)**
 - Precondition of S.op must be equal to or weaker than T.op's precondition
- **Covariance (output side)**
 - Postcondition of S.op must be equal to or stronger than T.op's post condition

SOLID



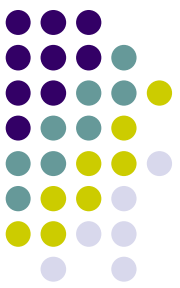
SOLID

Guidelines for structures that can survive shifting tides.



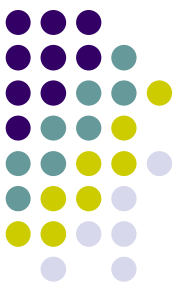
SOLID principle

- S.O.L.I.D. principle
 - mnemonic acronym
- **S**ingle responsibility
- **O**pen-closed
- **L**iskov substitution
- **I**nterface segregation
- **D**ependency inversion



Inheritance issues

- Understand the difference between class inheritance and interface inheritance (or subtyping).
 - **Class inheritance** defines an object's implementation in terms of another object's implementation.
 - mechanism for code and representation sharing.
 - **Interface inheritance** (or subtyping) describes when an object can be used in place of another.
- I hate those names!



Inheritance issues

Here's the problem they are both inheritance.... so what names should they have?

A) LinkedList is a base class...

- is implementing the derived class in terms of the base class

B) Dog is base class and Poodle is derived is inheritance...

- "is-A" or polymorphic.

Gang of Four

A is call Class Inheritance

B is called Interface Inheritance

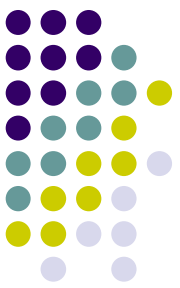
CONFUSING...

A is a Mix-In of only one class (LinkedList)

B is a Polymorphic class... the Derived is-A Base class.

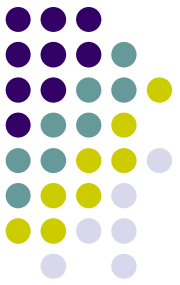
From now on.. I'm using Mix-In, and Polymorphic class as my description.

Mixin



- Mixins
 - language concept that allows a programmer to inject some code into a class
 - a style of software development, in which units of functionality are created in a class and then mixed in with other classes
- Mixin class acts as the parent class
 - containing the desired functionality.
 - Subclass can then inherit or simply reuse this functionality
 - but not as a means of specialization
- Mixin will export the desired functionality to a child class, without creating a rigid, single "is a" relationship.
 - Important difference between the concepts of mixins and inheritance,
 - child class can still inherit all the features of the parent class, but, the semantics about the child **"being a kind of"** the parent need not be necessarily applied

Thank You!



- Questions?

